

Spring Boot JWT Demo

Basic Spring Boot and Thymeleaf demo for JWT login.

Overview

- A login page sends username and password to the backend.
- The backend checks the user in MySQL.
- Passwords are stored as SHA-512 hashes, not plaintext. The backend hashes the submitted password and compares it to the stored hash.
- The backend generates a JWT after successful login.
- The JWT is saved to `generated-jwts.txt` for students to copy into jwt.io.
- The backend stores the JWT in an `HttpOnly` cookie for the browser.
- Thymeleaf renders the login, failure, and product pages on the server.
- Postman can call the product API with `Authorization: Bearer <jwt>`.
- Admin users can see all products. Normal users see only their own products.

Application Flow

1. The student opens the frontend login view at `/`.
2. Spring MVC renders `src/main/resources/templates/index.html`.
3. The login form submits username and password to `POST /login`.
4. The request reaches
`src/main/java/sg/edu/nus/secure_api/controller/LoginController.java`.
5. `LoginController.java` calls
`src/main/java/sg/edu/nus/secure_api/service/AuthService.java`.
6. `AuthService.java` looks up the user with
`src/main/java/sg/edu/nus/secure_api/repository/ProfileRepository.java`, hashes the submitted password with SHA-512, and compares it to the stored hash.
7. If the login is valid, `AuthService.java` calls
`src/main/java/sg/edu/nus/secure_api/service/JwtService.java` to generate a JWT.
8. `AuthService.java` saves the generated JWT into `generated-jwts.txt`.
9. `LoginController.java` stores the JWT in an `HttpOnly` cookie and redirects to `/products`.
10. Before `/products` reaches the controller,
`src/main/java/sg/edu/nus/secure_api/security/JwtAuthenticationFilter.java` checks whether the JWT cookie is valid.
11. If the JWT is valid, `ProductPageController.java` reads products and prepares the page model.
12. Spring MVC renders the final product view with `src/main/resources/templates/products.html`.
13. If login fails, Spring MVC redirects to `/login-failure` and renders
`src/main/resources/templates/login-failure.html`.

`ProductPageController.java` renders the Thymeleaf product page. `ProductController.java` is a REST controller for Postman/API access.

Where to Send and Extract JSON

These are the two endpoints where data crosses between the browser and the backend.

Purpose	Endpoint	Direction	Controller
Login (send credentials)	POST /login	Browser → Backend	LoginController.login(...)
Product API (extract JSON)	GET /api/products	Backend → Browser	ProductController.productsApi(...) (@RestController)

- **Send to the backend:** the login at POST /login is where the browser sends the username and password.
- **Extract from the backend:** the product API at GET /api/products is where the backend returns the product list as JSON (requires a valid JWT).

Diagram

Use Case Diagram



Class Diagram

```

classDiagram
class LoginController {
    -AuthService authService
    +loginPage() String
    +login(String username, String password, HttpServletResponse response)
String
    +logout(HttpServletResponse response) String
    +loginFailure() String
    -buildLoginCookie(String token) ResponseCookie
    -buildLogoutCookie() ResponseCookie
}
class AuthService {
    -JwtService jwtService
    -ProfileRepository profileRepository
    -Path jwtOutputFile
    -int jwtOutputMaxEntries
    +login(String username, String password) String
    -sha512(String value) String
    -saveTokenToLocalFile(String username, String token) void
}
class JwtService {
    -String secretKey
    -long expirationMillis
    -getSigningKey() SecretKey
    +generateToken(String username, String role) String
    +authenticateToken(String token) Claims
}
class JwtAuthenticationFilter {
    +String AUTH_USERNAME
    +String AUTH_ROLE
    +String AUTH_COOKIE_NAME
    -JwtService jwtService
    #doFilterInternal(HttpServletRequest request, HttpServletResponse
response, FilterChain chain) void
    -isProtectedPath(String path) boolean
    -resolveToken(HttpServletRequest request) String
    -resolveTokenFromAuthorizationHeader(HttpServletRequest request) String
    -resolveTokenFromCookie(HttpServletRequest request) String
    -handleUnauthorized(HttpServletResponse response) void
}
class ProfileRepository {
    +findByUsername(String username) Optional~Profile~
}
class ProductPageController {
    -ProductController productController
    +productsPage(String username, String role, Model model) String
}
class ProductController {
    -ProductRepository productRepository
    +productsApi(String username, String role) List~Product~
    +getProducts(String username, String role) List~Product~
}

```

```

class ProductRepository {
    +findAll() List~Product~
    +findByOwner(String owner) List~Product~
}
class Profile {
    -Long id
    -String username
    -String password
    -String role
    +getId() Long
    +getUsername() String
    +getPassword() String
    +getRole() String
}
class Product {
    -Long id
    -String name
    -String category
    -String description
    -String owner
    +getId() Long
    +getName() String
    +getCategory() String
    +getDescription() String
    +getOwner() String
}

LoginController ..> AuthService : uses
AuthService ..> JwtService : uses
AuthService ..> ProfileRepository : uses
JwtAuthenticationFilter ..> JwtService : uses
ProductPageController ..> ProductController : uses
ProductController ..> ProductRepository : uses
ProfileRepository ..> Profile : returns
ProductRepository ..> Product : returns
Product ..> Profile : owner username

```

Sequence Diagram (Login with Hashed Password)

```

sequenceDiagram
    actor User
    participant Browser
    participant LoginController
    participant AuthService
    participant ProfileRepository
    participant JwtService
    participant JwtAuthenticationFilter
    participant ProductPageController
    participant ProductController
    participant ProductRepository

```

```

User->>Browser: Enter username + password
Browser->>LoginController: POST /login
LoginController->>AuthService: login(username, password)
AuthService->>ProfileRepository: findByUsername(username)
ProfileRepository-->>AuthService: Profile with stored SHA-512 hash
AuthService->>AuthService: sha512(password)
AuthService->>AuthService: compare computed hash vs stored hash
alt profile exists and hashes match
  AuthService->>JwtService: generateToken(username, role)
  JwtService-->>AuthService: JWT
  AuthService->>AuthService: saveTokenToLocalFile(username, JWT)
  AuthService-->>LoginController: JWT
  LoginController-->>Browser: Set HttpOnly JWT cookie + redirect /products
  Browser->>JwtAuthenticationFilter: GET /products with authJwtToken cookie
  JwtAuthenticationFilter->>JwtService: authenticateToken(JWT)
  JwtService-->>JwtAuthenticationFilter: Claims(username, role)
  JwtAuthenticationFilter->>ProductPageController: continue request with
auth attributes
  ProductPageController->>ProductController: getProducts(username, role)
  alt role is ADMIN
    ProductController->>ProductRepository: findAll()
  else role is USER
    ProductController->>ProductRepository: findByOwner(username)
  end
  ProductRepository-->>ProductController: products
  ProductController-->>ProductPageController: products
  ProductPageController-->>Browser: render products.html
else profile missing or hashes do not match
  AuthService-->>LoginController: throw IllegalArgumentException
  LoginController-->>Browser: redirect /login-failure
end

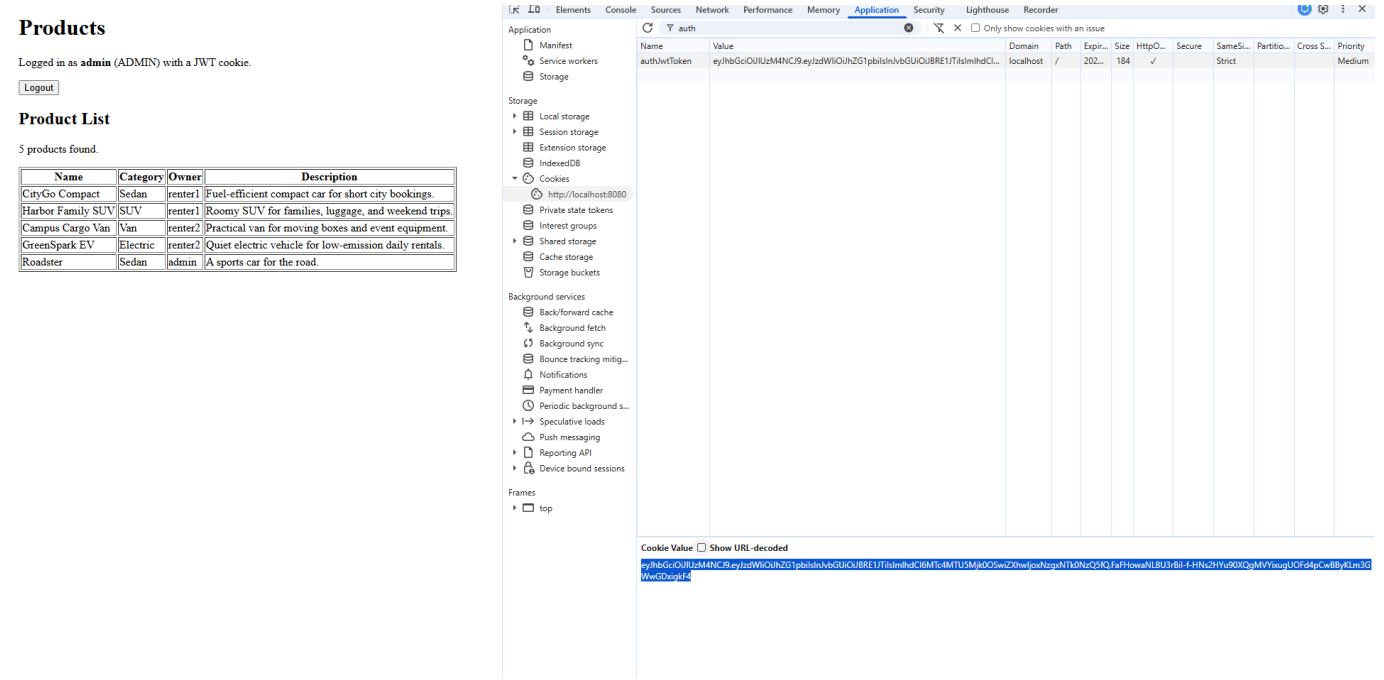
```

How to Get the JSON from the Product API

The `/api/products` endpoint only accepts the JWT in the `Authorization: Bearer` header (the browser cookie alone does not work for the API). So you must send the token in the header.

First, get a token:

1. Log in through the browser (for example `admin / admin123`).
2. Get the JWT in one of two ways:
 - Copy the latest JWT from `generated-jwts.txt`, or
 - Press `F12` to open DevTools, go to the **Application** tab, open **Cookies**, and copy the value of the `authJwtToken` cookie.



Then call the API with one of the methods below.

Method 1: Postman

- 1. Set the request to **GET** `http://localhost:8080/api/products`.
- 2. Open the **Authorization** tab, choose **Bearer Token**, and paste the JWT into the **Token** field.
- 3. Click **Send**.

```
GET http://localhost:8080/api/products
Authorization: Bearer YOUR_JWT_HERE
```

The JSON product list appears in the response body.

GET http://localhost:8080/a + No environment

http://localhost:8080/api/products Save Share

GET http://localhost:8080/api/products Send

Docs Params Authorization Headers 7 Body Scripts Tests Settings Cookies

Auth Type: Bearer Token

Token: j4pCwBByKLM3GWwGDxigkF4

The authorization header will be automatically generated when you send the request. Learn more about [Bearer Token](#) authorization.

Body 200 OK • 11 ms • 821 B

JSON Preview Visualize

```
1 [
2   {
3     "category": "Sedan",
4     "description": "Fuel-efficient compact car for short city bookings.",
5     "id": 1,
6     "name": "CityGo Compact",
7     "owner": "renter1"
8   },
9   {
10    "category": "SUV",
11    "description": "Roomy SUV for families, luggage, and weekend trips.",
12    "id": 2,
13    "name": "Harbor Family SUV",
14    "owner": "renter1"
```

Method 2: curl (PowerShell)

In PowerShell, use `curl.exe`:

```
curl.exe -H "Authorization: Bearer YOUR_JWT_HERE"
http://localhost:8080/api/products
```

The JSON is printed directly in the terminal.

Database Setup

Open MySQL Workbench and run:

```
database/init.sql
```

This creates:

- `profiles`
- `products`

Run

Set environment variables in PowerShell:

```
$env:DB_USERNAME="root"  
$env:DB_PASSWORD="YOUR_PASSWORD"  
$env:JWT_SECRET="use-a-long-demo-secret-key-at-least-32-characters"
```

Start the app:

```
.\mvnw.cmd spring-boot:run
```

Open:

```
http://localhost:8080
```

JWT Output File

After each successful login, the server writes the generated JWT to:

```
generated-jwts.txt
```

The file keeps only the latest tokens. Change this number in:

```
jwt.output-max-entries=5
```